

语义层是写的，记忆是挣的 The payload: the semantic layer, memory, and the loop that fills it

本篇范围说明：这是 [Neo4j 开发者博客上的一篇厂商技术文章](#)（发表于 Medium，是一个系列的第二篇——1A 讲架构主张、1B 搭线路，本篇讲「线路上跑的是什么」）。抽出的 38,211 字符里只有约 3 万字符是正文，其余是站点导航与产品菜单，已剔除。

与前几天的 arXiv 论文不同，这是一篇标准版权的公司博客，所以下面「全文翻译」一节按章节推进、摘引承载设计决定的关键句并译出，其余段落以中文浓缩转述，不做整篇逐段照抄。产品介绍段（AuraDB、Neocarta 等自家产品）只保留与设计判断相关的部分。文末原文链接与开源仓库地址在「参考链接」里给全。

核心内容

Q：这篇要解决什么问题？ A：作者开篇的问题陈述很准：

每一个有用的 AI agent 都会撞上同一堵墙：它在抽象层面很聪明，在具体事情上一无所知。

它会推理，但它不知道你那个 revenue 列到底扣没扣折扣、哪个客户的合同快到期了、昨晚的数据加载其实跑晚了、或者它正要写的那个 join 前三次有人试都是错的。

那些知识是存在的——在数仓的 schema 里、在业务术语表里、在运维手册里，而且大部分在人的脑子里——但它们不在 agent 够得着的地方。于是 agent 猜，每个会话都从零开始，并且重犯一个同事上周犯过的错。

Q：那这个「载荷」是什么？先说它不是什么 A：整个设计转在这一条规则上：

它不是数据。图不复制企业的存储系统里已有的东西。

分工是：企业系统保管数据；图保管指针、一个语义层（关于数据的元数据）、以及记忆（工作中挣来的 agent 侧元数据），并把它们提供给任何消费上下文的东西。

理由很实际：数据湖仓、数仓、业务库、文档库，数据就待在它现在待的地方，受治理、受监管，而且大到根本不可能迁进某个新的 agent 记忆存储里。图携带的是元数据，是 agent 要在数据上行动所需的操作性知识，而它一行数据都不必看到。

Q：元数据的两个家，差别在哪？ A：这是全文的骨架：

语义层

记忆

是关于什 么的	关于数据	关于工作
谁写的	被撰写的 (authored)	被挣来的 (earned), agent 自己在 干活时写
什么时候 变	只在数据平台变化时	每个会话都在变
会不会复 利	不会——术语表第一百天和第一天一样好, 而这是对的, 你 不想让定义漂移	会——记忆就该漂移, 而且是往上漂

作者用一句话收口: 「语义层是被撰写的。记忆是被挣来的。」

Q: 语义层具体装什么? 为什么 schema 不够? A: 作者的例子很具体: 一个叫 util_pct 的列、一张叫 acct_pr_usage 的表, 对模型来说什么可信的信息都没有。

一个只能盯着原始 DDL 的 agent, 会猜 join、猜一行代表什么、猜「active」是什么意思——而且会以看起来完全合理的方式猜错。

语义层是业务说话方式与数仓存储方式之间的翻译。它说: 这张表是按月的「客户-产品」使用率; 一行等于一个客户一个产品一个月; revenue 已经是扣除折扣后的净额; 这张表今早加载干净; 「at risk」的意思是九十天内到期续约且使用率在下滑。

这后面几句没有一句在任何 schema 里。

三种元数据挂在同一批节点上:

类型	内容	关键设计
技术元数 据 (骨架)	库 → schema → 表 → 列; 列带类型、可空性、键标记	一列是另一列的外键时, 目录把它画成一条引用边——于是那个 agent 本来要猜的 join, 已经被写下来了; 每列还挂几个示例值, 让模型在过滤前就能看到 status 字段实际写的是 active 还是 churned
业务元数 据 (含义)	每张表每列一句人话描述	业务术语被做成它们自己的节点, 各自打标到它适用的表和列上——于是「at risk」或「净收入」在图里是一等公民, 而不是埋在注释里的一个短语
操作元数 据 (下一步)	新鲜度、上次何时加载、是否加载干净、健康度	回答的是「agent 现在能不能信这张表」, 而不只是「它是什么」

因为含义和结构活在同一批节点上, 目录不需要第二套系统就能被搜索: 描述进向量索引, 名称、描述、术语进全文索引。「持有一张表含义的那些节点, 就是你用来检索到它的那些节点。」

Q: 它带来的那个「反转」是什么? A:

agent 先遇到含义，后遇到结构。

它用业务词汇搜目录，语义层把那些词解析成物理表和列，**然后才**构建查询、通过数据源工具去跑。作者的口号是 Find first, then fetch（先找到，再取数）。

图引导这次读取，但从不持有数据的副本。

另外，**统一形状本身就是目的**：不是每个系统一条定制检索路径，而是**每个企业数据源都被投影成同一个表示**，这个表示成为数据检索的唯一真相源。agent 不用问「我那 N 个系统里哪个可能有续约数据」然后猜，它直接问目录「续约数据在哪」。

Q：记忆那一半是什么形状？ A：一条流进来，几个消费者出去。

会话里的每一个事件——提示词、工具调用、结果、失败——**在发生的当下就被记进一条有序的流里，此时还没有任何模型判断什么重要。**

捕获是被动的、确定性的、完整的，而思考发生在后面。

「后面」指的是**离开当前这一轮**：在 Stop 时触发，并做限流，所以它永远不阻塞你。后台的几遍处理去读那份原始语料。区别在于每一遍向它提的问题不同，一共三问：发生了什么、哪里错了、什么值得记住。

Q：第一遍（时间线） A：无条件的。每个工作窗口变成一到三个 `Observation` 节点挂在项目时间线上：一个类型（变更/修 bug/特性/重构/发现/决定/问题）、一句标题、几条具体且自包含的事实、以及两到四句叙述（被问了什么、做了什么、怎么收场）。用 `NEXT` 串起来，项目在头部保一个指针，**于是时间线可以按顺序走。**

追加式、从不设闸门、从不去重、并且被嵌入所以可搜索。

理由说得很好：「一个项目的历史，不该由一个模型来决定什么有意思、然后由它来策展。」

Q：第二遍（工具失败 → 每工具的模式） A：这一遍是全文工程含量最高的部分。

捕获是确定性的、且在带外：每次 agent 通过工具跑查询，一个钩子检查结果，**不占用模型那一轮的任何东西**，并从响应的文本和形状分类失败，路径里没有模型。

它区分：权限失败、超时、结果大小与资源限制、能力缺失、schema 漂移、纯语法/解析错误。**而且它抓两种根本不抛异常的「安静失败」**：查询成功但返回零行；以及一个本该投影出时间或空间值的位置回来了一个空对象。

每个变成一个 `:QueryExecution` 节点，带类型化的 `:QueryIssue` 子节点。

一处跨 harness 的现实：Claude Code 会触发一个实时的 `PostToolUseFailure`，而 Codex 是在普通 `PostToolUse` 上返回一个错误形状的结果。**而且它在两个都不触发的 harness 上也能活**——Stop 时那一遍会重读记录并从中重建失败的查询，**收敛到同一批节点 id 而不是产生重复。**

固化 (consolidation) 把证据变成指导：一个限流的服务把未固化的失败折进 `:QueryErrorPattern` 节点，每个带规范化的错误签名、根因、可执行的修法、以及在可能时一份改正后的查询。

这就是「这条查询失败了」和「这种形状的查询会因为 X 失败，改用 Y」之间的差别。

为什么要固化而不是存原始日志：「指望模型在召回时读一堆堆栈跟踪来推出教训，是在要求昂贵的那条路去重做便宜的那条路本来做一次就够了的事。」

Q：它拒绝固化什么？这一段是全文最好的 A：「它拒绝固化什么，和它固化什么一样重要。」

只有**确定性地由查询文本本身造成**的失败才有资格：语法错误、schema 不匹配、缺失的过程与函数、序列化投影问题。

超时、限流、资源耗尽、权限错误被排除了两次——一次靠硬编码的问题类别让它们根本进不了队列，另一次在提示词里指示模型丢掉藏在通用错误桶里的一次性瞬态问题。

理由是这句：

超时是关于那个下午的事实，不是关于这条查询的事实。固化它会教给下一个 agent 一个迷信，而一个记忆库里的迷信比一个空的记忆库更糟，因为它自信地错着，而且会复利。

模式是按工具锚定的，不是按项目：每个曾经失败过的工具在项目下挂一个 `:ToolErrorProfile`，模式挂在 profile 下，每个模式用 `DERIVED_FROM` 链回它被提炼自的那些执行记录，于是**每一条指导都能追溯到挣出它的那些失败**。

分组的意义：「BigQuery 的方言怪癖不是给 Cypher 失败的建议。」

Q：召回怎么闭环？ A：下次同一个工具因为非法查询失败时，召回钩子搜索这个工具自己的模式库，同时在失败的查询文本和错误消息两侧做混合匹配，打向量索引和全文索引、用倒数排名融合，并且永远限定在出错的那个工具范围内。

两侧都搜是它在两个方向都能work 的原因：**错误看起来眼熟时能找到，查询看起来眼熟但错误措辞不同时也能找到**。

取回的修法**在它失败的同一轮里**就被递回给模型，在它开始盲目重试一个根本不可能成功的查询变体之前。

闭环发生在这一轮之内，而不只是跨会话。撞上错误的那个 agent，就是拿到修法的那个 agent。

一处诚实的实现细节：`PostToolUse` 支持返回 `additionalContext`，那是把文本放到模型面前的干净通道。而 `PostToolUseFailure` 不支持——它文档化的回传通道是一个顶层的 `decision: "block"`，其 reason 会被展示。所以钩子去「阻止」一个已经失败了的工具调用，纯粹是为了把修法塞进上下文。什么都没有被真的阻止。作者的评语：「这是对一个控制流动词的小小滥用，而它是墙上唯一的那扇门。」

这个钩子还泛化出一个用途：有些工具不返回可分类的错误，而是返回**值得留存的结构化事实**。在他们的演示里，实体解析和新闻检索工具返回结构化 JSON，钩子把它拆开直接并进领域图——**以外部系统的真实 id 为键**，于是同一个实体被看到两次会落在同一个节点上而不是产生重复。

效果是：**一次外部查询只付费一次**。第一个问到这个客户的会话调 API，答案作为持久可查询的结构写进图；之后每个会话都从图里读，**API 调用从习惯变成了例外**，而那份富化**不再随着取来它的那次对话一起蒸发**。

Q：第三遍（持久学习）有什么约束？ A：可复用的事实、环境怪癖、持久偏好、任务模式都变成 `:Learning` 候选。

每个候选带的是一个动作而不只是文本：新建、更新一个已有的且实质说同一件事的、或者**忽略**那些例行且瞬态的工作。「**最后那个动词承载了整个设计，因为这个库是要固化而不是堆积。**」

两条约束比看起来更重要：

一、抽取器读对话文本，从不读工具输出。

这是一条安全边界，不是性能上的调优。

理由：工具结果是攻击者可触达的，而用户自己说的话不是。**一个被投毒的网页、一行恶意数据、或一份被篡改的 API 响应，不该仅仅因为被返回给了模型，就能把自己写进持久记忆。**

结构化的工具结果**确实**会进图（就是上面那条富化路径），但它们**以带真实 id 和已知来源的类型化节点的形式到达**——在那里一个恶意值是一行你能找到并删掉的坏数据，而不是一条 agent 当成自己信念的自由文本。

二、抽取提示词是代码里的一个固定常量。

改变抽取器找什么，意味着**刻意地、在一次提交里**编辑那个模板。

运行时什么都不调它，也不该调，因为一个凭自己输出的强度来重写自己的提示词，是一个任何地方都没有 ground truth 的反馈回路，而它的失效模式是安静且缓慢的。

Q：用户域和项目域怎么分？ A：「不是 agent 学到的每一件事，都属于它学到它的那个仓库。」

	用户域	项目域
内容	关于这个人的持久事实：姓名、角色、偏好简短回答或英式拼写、总是想先看查询成本、某个反复出现的约束或长期优先级	关于数仓、领域、代码库的事实
该怎样	应该跟着他进入下一个项目，而不是随这个项目一起死掉	留在原地
何时注入	会话开始时 ，因为它在你打任何字之前就为真	针对你刚发出的那条提示 ，因为它的相关性取决于你在问什么

抽取器在写的时候就做这个判断：只有在信号确实含糊时才默认归项目；一旦信号明确是关于这个人的，就在第一次提及时立即归到人，不等它重复。只有在推断边缘特质（而不是本人明说的）时才要求重复。

作者收了一句：「一个库，两种寿命。」

Q：两道闸门是怎么设计的？ A：抽取从不直接写记忆，理由是：

一个把自己做过的每件事都存下来的 agent，会像固化它的胜利一样忠实地固化它的错误，而频率不等于正确性。

第一道闸门是自动的，而且它妙在不是在旁边另搭一套系统：每条学习在被写下的那一刻就被嵌入（这正是召回后来能按含义找到它的原因），而这道闸门只是把那次搜索掉转向内。它嵌入新候选、取回同项目同域里的最近邻，然后问一个大模型裁判两个很窄的问题：这条是否真的与已持有的某条矛盾？它是否只是已持有内容的一次重述？

结果分五种：干净的通过；重述折进它所重复的那一条并强化它，而不是堆在它旁边；赢家取代输家；一个证据充分的在位者否决一个推测性的新来者；真正含糊的被原样留下，而不是去猜。

被拒绝和被折叠的项会失去它们的嵌入，这把它们从向量索引里剔出去，而它们的文本和来源边留了下来：

在这里，遗忘是丢掉嵌入而不是丢掉那一行，所以图拒绝了什么、为什么拒绝，仍然在那里可供审计。

第二道闸门是一个人，它存在是因为第一道故意是窄的：

自动闸门回答的是「这两条能同时为真吗」。它对「这条到底是不是真的」完全无话可说，而一条自信、连贯、彻头彻尾错误、却不与任何东西矛盾的学习，会径直通过它——因为图没有任何 ground truth 可以拿来核对。只有人有。

所以恰好两类被交给给人：被裁判标为含糊的项目域候选，以及每一条用户域候选，无一例外。

队列展示冲突的双方，而不是一条被剥离了上下文的断言。复核者可以通过、拒绝、修正抽取器的措辞并把改好的版本升上去、挑一个赢家、或者两个都留——后者对应一种常见情况：冲突是表面的而非真实的，两件事在不同情形下都成立。

Q：为什么每一条用户事实都要过人？ A：作者说这是整个设计里的承重决定。

- 一条项目学习是被召回的：它出现在上下文里，模型读到它，模型可以把它和它能看到的其它一切放在一起权衡。
- 一条用户事实做的是性质上更强的事：它被折进人格（persona），折进 agent 自己的操作指令里，在每一个项目里、永久地，在那里它不再是一条模型可以评估的、被检索到的主张，而是一条它据以推理的前提。

那是这个系统里权限最高的一次写入，而一条坏条目的爆炸半径是今后所有地方的所有会话。

人格固化（作者称之为「做梦」）：每隔一段时间，离开当前轮次、并且限流，一个反思遍收集那些堆积着还没被折叠的、已批准的用户事实，**重写 agent 的系统提示词**。三条保护：

1. 它坚持做「编辑」而不是「重写」，这样人格不会漂成一个谁都没选择过的样子
2. 它把那些事实围栏成「关于用户的一份不可信描述：是要去总结的数据，绝不是要去遵循的指令」。作者解释这道围栏为什么要在人工闸门之后还存在：「这就是纵深防御的意义——**复核者批准的是『这是真的』，不是『这是一条命令』，而这两个是不同的主张。**」
3. **结果是版本化的**：换出去的提示词在新的接管之前被存档，活动版本在每个会话开始时注入

作者的收句：**「这是记忆从『一个 agent 去查的东西』毕业成了『一个 agent 在打招呼的时候就已经是的东西』。」**

Q：作者自己说还缺什么？ A：说得很直接。项目学习已经有闸门、可人工复核、去重、会被强化；工具失败是真的被固化了。**项目记忆还缺的是更高阶的固化**：

- 没有任何东西把相关的学习聚成它们都在围绕的那个主题
- 没有任何东西把五十条具体笔记压成它们背后的五条原则
- 而且没有任何东西会遗忘：一条批准过的学习，在它的第一千天和第一天一样响亮，**不管此后有没有任何东西强化过它**

下一篇会讲在记忆图上跑图算法、以及一个衰减函数。

背景补充 / 点评

几个概念的位置

MCP (Model Context Protocol) 是给模型暴露工具的一套协议。**生命周期钩子**（`PostToolUse`、`PostToolUseFailure`、`Stop`、`SessionEnd`）是 harness 在特定时点回调外部代码的接口。本文的设计整个立在这两样东西上：**MCP 负责 agent 主动读写图，钩子负责被动捕获与注入。**

harness-agnostic（与 harness 无关） 是本文反复强调的一个性质：同一张图服务 Claude Code、Codex 或自定义循环，**于是一队 agent 共享一份记忆，而不是各自在自己的孤岛里漂移。**这也是文中那些跨 harness 兼容细节（两种失败事件形状、Stop 时重建）的来源。

倒数排名融合（RRF） 是把多路检索结果合并的一种简单办法：按每一路给出的排名倒数加权求和。本文用它融合向量索引和全文索引的结果。

claude-mem 是本文时间线那一遍借鉴的开源项目：把一次会话的工作捕获成追加式的观察记录，在下一次会话开始时回放。作者说保留了机制、但把目标从「开发者的文件和编码会话」换成了「在数据和领域上工作

的 agent」。

这是一篇厂商文章，要先说清楚。语义层那一半的实现（连接器、目录）是 Neo4j 的产品（文中叫 Neocarta），记忆那一半是开源的（[neo4j-labs/meta-knowledge-graph](#)）。设计思路和产品是可分离的，作者大体也分开了，但读的时候要知道哪一半你能直接拿来用。

这篇最值得学的五处

一、「语义层是被撰写的，记忆是被挣来的」——一句话把两种元数据的治理方式分开了。

它们的差别不只是内容，而是该不该漂移：术语表不应该漂（定义漂移是 bug），记忆应该漂（而且是往上漂）。把这两种东西放在同一个维护流程里，一定会在某一侧出错。

二、「捕获是被动的、确定性的、完整的，思考在后面。」

一条有序流无条件进来，此时不做任何「什么重要」的判断；三遍后台处理各自向它提一个不同的问题。这个分离的真正好处文中没有明说但很明显：[判断的标准变了，可以重跑那一遍；而如果捕获时就做了判断，原始信息已经没了。](#)

三、「超时是关于那个下午的事实，不是关于这条查询的事实。」

这是我读过的、对「什么样的经验才配进知识库」最好的一句判据。它背后的完整论证是：[一个记忆库里的迷信比一个空的记忆库更糟，因为它自信地错着，而且会复利。](#)而实现上它排除了两次（硬编码类别 + 提示词指令），这是对待不该进库的东西的正确姿态。

四、「自动闸门回答『这两条能同时为真吗』，它对『这条是不是真的』完全无话可说。」

这是我见过对「一致性检查能做什么、不能做什么」最锋利的表述。而且他们给出的对策不是把闸门做得更聪明，而是[承认它的边界，然后把恰好两类东西路由给人。](#)

五、「遗忘是丢掉嵌入，而不是丢掉那一行。」

被拒绝的东西从检索索引里掉出去，但文本和来源边留着[可供审计](#)。这是一个很小但很对的工程决定：[「这条为什么没进来」](#)是一个会被反复追问的问题，而删掉行就永远答不了了。

四处该打折扣的地方

一、全文没有任何一个数字。

没有任何测量：记忆层到底帮了多少、和没有记忆比是什么差距、闸门的误判率是多少、那个大模型裁判判对的比例是多少——[一个数都没有。](#)

页头那条横幅引用的「GraphRAG 让 AI agent 真实性提高 80%」是[另一份（外链的）报告](#)，不是这套设计的证据。

对一篇关于「什么该进记忆」提出了如此强主张的文章，完全没有测量是最大的缺口。对比今天读的另一篇（Clinical Graph-JEPA），后者至少有一个受控的 A/B 和消融。

二、「确定性地由查询文本本身造成」这条判据是一个硬编码清单，而且没被检验过。

排除超时、限流、资源耗尽、权限错误——这个方向是对的。但**「哪些失败是由查询文本确定性造成的」本身是一个判断，被编码成了一份固定清单**，而文中没有任何关于这份清单在两个方向上出错频率的测量（真正的查询缺陷被归成了瞬态、或者反过来）。文章断言这条边界是干净的，但没有展示它是干净的。

三、人格那条路径有一个被点名却没有关掉的单点。

作者很清醒：自动闸门判不了真假，一条连贯但错误的学习会径直通过。他们的答案是**每一条用户域候选都要人复核**。对正确性而言这是对的答案，但它让整个系统里价值最高的那部分变成了由一个人限流的——而文中没有讨论队列没人看的时候会怎样：人格就此停止改进？候选会过期？

四、「没有任何东西会遗忘」和「重述会强化」这两条合起来是一个棘轮，而这层交互没有被点出来。

作者点名了缺衰减这个缺口。但**折叠重述会「强化」在位者**，再加上没有衰减，意味着一条早期被重述过几次的学习会永久地响亮下去，而世界变了它也不会变轻。只提衰减缺失，不提「强化 + 无衰减 = 棘轮」，是少说了一半。

另外那个「在失败的查询文本和错误消息两侧都做混合匹配」是个相当好的想法，**只拿到一段篇幅**——它的洞察（错误眼熟时能找到，查询眼熟但错误措辞不同时也能找到）值得更多。

对我的启示

一、「语义层 vs 记忆」这个二分，正好切开我的知识图谱里一直混着的两样东西。

	我这边对应什么	该不该漂移
语义层（被撰写的）	K1 - K8 / S1 - S12 的域定义、准入判据、行为归并表、comparator 口径	不该。判据漂移就是 bug
记忆（挣来的）	标注中发现的判据缺口、两人分歧的裁定结果、「这一类语义点总被误判成 K3」这种经验	该，而且该往上积累

现在这两样活在一个地方（spec 文档 + note 字段），由同一个流程维护。于是「绝不该漂的」和「就该往上漂的」用的是同一套规矩。这是一个我此前没有命名过的结构问题。

二、「捕获无条件、判断在后面的单独一遍」——这条把我前两天记的教训变成了可执行的做法。

我昨天记的是「只留判决、不留路径，会把过程里已有的信息销毁掉」。这篇给了正确的形状：**原始流无条件全量记下，判断放进一个可以单独重跑的后置遍**。

关键收益是最后半句：判据一改，重跑那一遍即可，不需要重新标注原始数据。这正是我前几天那 25 条判据违例所缺的东西——在一趟过的架构里它们只能靠回标去修。

三、「超时是关于那个下午的事实」——我需要的正是这条判据的对应物。

我这边的同构问题是：**标注分歧该不该变成判据修订？**

- 由**真实的判据缺口**造成的分歧 → 该固化成修订
- 由一个标注者当时看漏了、或理解偏了造成的分歧 → **不该**

我现在对所有分歧一视同仁，全进裁定队列、都可能进修订清单。**照这篇的做法我需要一条排除规则：只有能确定性地追溯到判据文本本身的分歧，才有资格变成修订。**否则我是在把迷信编进判据——而「记忆库里的迷信比空的更糟，因为它自信地错着，而且会复利」这句话，对判据同样成立。

四、「抽取器读对话文本、从不读工具输出」是安全边界——这条对我的跨企业图谱是直接结论。

原文的理由是工具结果攻击者可触达而用户的话不是。对应到我紧要的场景：**如果企业 A 的标注产物能写进共享的 T-Box，那么 A 就能污染 B。**

共享层必须是「被撰写的」，不能是「被挣来的」。而这恰好和昨天那篇联邦图谱论文说的是同一件事（共享 T-Box 只含关系公理、无实例无三元组）。**两个独立来源现在指向同一条设计规则，这条可以定了。**

五、「提取提示词是代码里的固定常量」——这条我现在没破，但要写下来免得以后破。

我的判据就是我的抽取提示词。它应该只通过**裁定之后的刻意提交**改变，绝不能由流水线自己发现一个模式然后自我调整。

原文那句是理由：**一个凭自己输出的强度来重写自己的提示词，是一个任何地方都没有 ground truth 的反馈回路，而它的失效模式是安静且缓慢的。**现在没人提议这么做，但「让模型自动提出判据修订」是一个很容易被提出来的优化——**提前记下为什么不做。**

六、「自动闸门回答不了『这条是不是真的』」——这是第三个独立来源了。

- 前天：FAIR 数字对象那篇的「验证层次错位」（检查 URI 的域名而不是 URI 背后的实体）
- 昨天：我自己在 memory 里记的「一致率检查的是两人一不一致，不是标得对不对」
- 今天：这一篇

三个来源同一件事，可以当定论用了。而这篇还多给了一条可抄的策略：**把恰好两类路由给人，并且让权限最高的写入必须经人。**

对应到我：**判据修订是我这套系统里权限最高的写入**——它影响所有已标和未标的数据，爆炸半径是全部。所以它必须经人裁定，不能由一致率自动通过。这条要写进标注 SOP。

七、「遗忘是丢掉嵌入而不是丢掉行」——我现在是碰巧做对的，该把它变成设计。

被判否的语义点，我保留了（后来那 205 条 JP 判否语义点里有 151 条被回收成了技能示范语料，正因为没删）。**但那是运气，不是设计。**

该写进 spec 的表述是：准入判否 = 不进节点索引，不等于不进库。判否的条目连同判否理由一起留着，因为「这条为什么没进来」是一个会被反复追问的问题。

八、「强化 + 无衰减 = 棘轮」是一个我在动手之前就该记下的坑。

如果我以后加入「被更多企业提到的知识点权重更高」这类强化机制，没有衰减的话，早期标注的那几家企业会永久占据主导。这篇点名了衰减缺失却没点名这层交互，我在建之前先记下来。

文中金句

"Every useful AI agent runs into the same wall: it is brilliant in the abstract and ignorant in the specifics." 每一个有用的 AI agent 都会撞上同一堵墙：它在抽象层面很聪明，在具体事情上一无所知。

"That knowledge exists ... but it is not where the agent can reach it, so the agent guesses, starts every session cold, and repeats a mistake a colleague made a week ago." 那些知识是存在的……但它们不在 agent 够得着的地方，于是 agent 猜，每个会话都从零开始，并且重犯一个同事上周犯过的错。

"The semantic layer is authored. Memory is earned." 语义层是被撰写的。记忆是被挣来的。

"An agent left staring at raw DDL will guess the join, guess what one row means, guess what active means, and guess wrong in ways that look entirely plausible." 一个只能盯着原始 DDL 的 agent，会猜 join、猜一行代表什么、猜「active」是什么意思——而且会以看起来完全合理的方式猜错。

"Capture is passive, deterministic, and complete, and the thinking comes later." 捕获是被动的、确定性的、完整的，而思考发生在后面。

"a project's history should not be curated by a model deciding what was interesting" 一个项目的历史，不该由一个模型来决定什么有意思、然后由它来策展。

"a timeout is a fact about the afternoon, not a fact about the query. Consolidating it would teach the next agent a superstition, and a superstition in a memory store is worse than an empty one, because it is confidently wrong and it compounds." 超时是关于那个下午的事实，

不是关于这条查询的事实。固化它会教给下一个 agent 一个迷信，而一个记忆库里的迷信比一个空的记忆库更糟，因为它自信地错着，而且会复利。

"The agent that hits the error is the agent that gets the fix." 撞上错误的那个 agent，就是拿到修法的那个 agent。

"an agent that simply saved everything it did would consolidate its mistakes as faithfully as its wins, and frequency is not correctness" 一个把自己做过的每件事都存下来的 agent，会像固化它的胜利一样忠实地固化它的错误，而频率不等于正确性。

"The automatic gate answers 'can these two both be true'. It has nothing whatsoever to say about 'is this true at all', and a confident, coherent, entirely wrong learning that contradicts nothing sails straight through it, because the graph has no ground truth to check it against. Only a person does." 自动闸门回答的是「这两条能同时为真吗」。它对「这条到底是不是真的」完全无话可说，而一条自信、连贯、彻头彻尾错误、却不与任何东西矛盾的学习，会径直通过它——因为图没有任何 ground truth 可以拿来核对。只有人有。

"forgetting, here, is dropping the embedding rather than the row, so what the graph rejected and why is still there to audit" 在这里，遗忘是丢掉嵌入而不是丢掉那一行，所以图拒绝了什么、为什么拒绝，仍然在那里可供审计。

"a prompt that rewrites itself on the strength of its own output is a feedback loop with no ground truth anywhere in it, and its failure mode is silent and slow" 一个凭自己输出的强度来重写自己的提示词，是一个任何地方都没有 ground truth 的反馈回路，而它的失效模式是安静且缓慢的。

"the reviewer approved a fact as true, not as a command, and the two are different claims" 复核者批准的是「这是真的」，不是「这是一条命令」，而这两个是不同的主张。

全文对照（按章节推进）

开篇：问题与 MKG 是什么

原文以那堵「抽象上聪明、具体上无知」的墙开场（金句已在上面给出），指出所需知识散落在 schema、术语表、运维手册和人脑里，**而不在 agent 够得着的地方。**

Meta Knowledge Graph (MKG) 被定义为一个共享的、图结构的、面向 AI agent 的上下文层。它通过整套 harness 线路到达 agent：一个 MCP 服务器暴露 agent 读写图所调用的工具，加上生命周期钩子来捕获发生了什么、并在模型作答前注入相关内容。

It does one job: give whatever is consuming context the operating knowledge it needs to act correctly, and capture what it learns so the next consumer starts further ahead than the last.

它只做一件事：给任何正在消费上下文的东西（一个 agent、一个副驾、一个 BI 工具、一个应用）以正确行动所需的操作性知识，并捕获它学到的东西，好让下一个消费者比上一个起步更靠前。

它按设计与 harness 无关：同一张图服务 Claude Code、Codex 或自定义循环，**于是一队 agent 共享一份记忆，而不是各自在自己的孤岛里漂移。**

本篇在系列中的位置：1A 做架构论证，1B 搭线路（工具与钩子），**本篇讲线路上跑的是什么。**

载荷首先不是什么

And the first thing to say about the payload is what it is not. It is not the data. ... the graph does not copy what the enterprise's stores already hold.

关于这个载荷，第一件要说的是它不是什么。它不是数据。……图不复制企业的存储系统里已经有的东西。

Enterprise systems keep the data; the graph holds pointers, a semantic layer (most of the metadata, about the data), and memory (the agentic metadata learned by working), and serves them to whatever consumes context.

企业系统保管数据；图保管指针、一个语义层（大部分元数据，关于数据的）、以及记忆（工作中挣来的 agent 侧元数据），并把它们提供给任何消费上下文的东西。

后接理由：数据受治理、受监管、且大到不可能迁移；图携带的是**操作性知识，而它一行数据都不必看到。**这段以那句划分收尾：**语义层是被撰写的，记忆是被挣来的。**

语义层

这一节先论证 schema 不够（`util_pct` / `acct_pr_usage` 的例子，金句已给），再给出语义层实际承载的那几句业务事实，并强调**这后面几句没有一句在任何 schema 里。**

随后介绍他们的实现（Neocarta，属自家产品）：连接器内省数据源、把目录以**统一形状**写进图、并把描述嵌入**使目录可按含义检索**。这一段的设计判断是：

That uniformity is the whole point. Instead of one bespoke retrieval path per system, every enterprise source is projected into a single representation, and that representation becomes the

single source of truth for data search.

那种统一性本身就是全部意义所在。不是每个系统一条定制的检索路径，而是每个企业数据源都被投影成同一个表示，而那个表示成为数据检索的唯一真相源。

That is the inversion the semantic layer buys you: the agent meets meaning first and structure second. ... Find first, then fetch.

这就是语义层给你买来的那个反转：agent 先遇到含义，后遇到结构。……先找到，再取数。

The connector builds the graph; the MCP tool reads it and hands the agent a pointer plus context. The agent reasons, then your own custom tools read the enterprise data directly. The graph guides the read but never holds a copy of the data.

连接器构建图；MCP 工具读取它，递给 agent 一个指针加上下文。agent 推理，然后你自己的定制工具去直接读企业数据。图引导这次读取，但从 not 持有数据的副本。

接下来三段分别讲技术元数据（骨架、外键画成引用边、示例值）、业务元数据（术语作为一等节点）、操作元数据（新鲜度与可信度，是目录的下一步）。这一节以检索的统一性收尾：

And because meaning and structure live on the same nodes, the catalog is searchable without a second system. ... the nodes that hold what a table means are the same nodes you retrieve it by.

而因为含义和结构活在同一批节点上，目录不需要第二套系统就能被搜索。……持有一张表含义的那些节点，就是你用来检索到它的那些节点。

记忆：一条流进，几个消费者出

这一节先说明记忆为什么不能由目录撰写——它不是关于数据的，是关于工作的：哪个工具答过这类问题、上次哪个 join 错了、哪次查询返回空以及为什么、用户纠正了什么、续约风险试了三次之后最终对的那个粒度。

It is also the only part of the payload that compounds. A glossary is as good on its hundredth day as its first, and that is correct, you do not want your definitions drifting. Memory is supposed to drift, upward, every corrected answer a note the next agent reads before it acts.

它也是整个载荷里唯一会复利的部分。一份术语表在它的第一百天和第一天一样好，而这是对的，你不想让你的定义漂移。记忆则理应漂移，而且是往上漂——每一个被纠正的答案，都是下一个 agent 在行动之前会读到的一条笔记。

然后是那个形状：一条流进来、几个消费者出去，捕获时不做判断（金句已给），三遍后台处理各问一个不同的问题。

三遍的内容（时间线、工具失败、持久学习）的完整机制、节点类型、闸门设计与人格固化，已在「核心内容」中按小节展开，此处不重复。其中承载设计判断的原文句子均已收入「文中金句」。

收尾：还缺什么

Put the three passes together and the write side has a shape. The timeline records what happened, unconditionally ... Tool failures distill into per-tool patterns that are replayed the moment the same tool fails the same way. And durable learnings are earned rather than accumulated: scoped to the person or the project, checked mechanically for contradiction, reviewed by a human wherever a machine has no standing to decide, and folded into the persona only once someone has said yes.

把三遍放在一起，写入这一侧就有了形状。时间线无条件地记录发生了什么……工具失败被提炼成按工具组织的模式，并在同一个工具以同样方式失败的那一刻被回放。而持久的学习是挣来的而不是攒来的：限定到人或项目，机械地检查矛盾，在机器没有资格决定的地方由人复核，并且只有在有人说了「是」之后才被折进人格。

What project memory still lacks is higher-order consolidation. Nothing clusters related learnings into the theme they are all circling. Nothing compresses fifty specific notes into the five principles behind them. And nothing forgets.

项目记忆还缺的是更高阶的固化。没有任何东西把相关的学习聚成它们都在围绕的那个主题。没有任何东西把五十条具体笔记压成它们背后的五条原则。而且没有任何东西会遗忘。

作者给的一句话总结：时间线记录一切；工具失败教会撞上它们的那同一个会话；而在通过机械检查或经过人之前，没有任何东西成为持久记忆。一张图承载所有这些。

参考链接

- [原文 \(Neo4j 开发者博客 / Medium\)](#) — 本篇原文，系列第二篇
- [neo4j-labs/meta-knowledge-graph](#) — 记忆那一半的开源实现：生命周期钩子捕获每个会话、MCP 工具召回项目记忆、大模型抽取循环提炼持久学习并演化 agent 的系统提示词。**语义层那一半（连接器与目录）属于自家产品，不在这个仓库里**
- [claude-mem](#) — 时间线那一遍借鉴的项目：把会话工作捕获成追加式观察、在下次会话开始时回放。本文保留了机制、换了目标域
- [Model Context Protocol](#) — 本文中 agent 主动读写图所走的协议
- [倒数排名融合 RRF](#) — 把向量检索与全文检索结果合并的办法，本文用于在「失败查询文本」和「错误消息」两侧同时匹配